

Global Engineering Solutions

FEST[®]

FEST Library Reference Manual

1.	Open		Page3
2.	GetSize		Page4
3.	GetLineLen		Page5
4.	Line		Page6
5.	GetFileName		Page7
6.	IsEOF		Page8
7.	ReadChar		Page9
8.	Read		Page10
9.	ReadLine		Page11
10.	ReadLineEx		Page12
11.	ReadComplexLine		Page13
12.	ReadBlock		Page15
13.	WriteChar		Page16
14.	Write		Page17
15.	WriteLine		Page18
16.	WriteFormat		Page19
17.	WriteFormatV		Page20
18.	Flush		Page21
19.	Find		Page22
20.	CopyFrom		Page23
21.	CopyTo		Page24



TFileA

1. Open

● Description

- file open 함수.

● Syntax

```
bool Open( const char* szPath , int mode );
```

● Parameter

- szPath : file 경로
- mode : file open 모드
(O_READ/O_READPLUS/O_WRITE/O_WRITEPLUS/O_APPEND/O_APPENDPLUS)

● Return

- true: file open success
- false: file open fail

● Sample

```
TFile file;  
file.Open( "Device/Robot.cfg" , TFile::O_READ);
```

2. GetSize

- **Description**

- Open 한 파일의 용량(byte) 확인

- **Syntax**

- int GetSize() const;

- **Parameter**

- Nothing

- **Return**

- Open 된 파일의 용량(byte)

- **Sample**

```
TFile Test;  
int isize;  
if(Test.Open( _TEXT("JobHistory.cfg") , TFile::O_READ ) ) {  
    isize=Test.GetSize();  
}
```

3. GetLineLen

● Description

- Open 된 파일의 가장 긴 Line의 Length를 확인하는 함수

● Syntax

- int GetLineLen() const;

● Parameter

- Nothing

● Return

- Line 의 Length

● Sample

```
TFile Test;  
int ilen;  
if(Test.Open( _TEXT("JobHistory.cfg") , TFile::O_READ  ) ) {  
    ilen=Test. GetLineLen();  
}
```

4. Line

● Description

- Open 된 파일의 line count를 확인 하는 함수

● Syntax

```
int Line() const;
```

● Parameter

- Nothing

● Return

- Line 의 count

● Sample

```
TFile Test;  
if(Test.Open( _TEXT("JobHistory.cfg") , TFile::O_READ ) ) {  
    printf("file line count=%d\n", Test.Line());  
}
```

5. GetFileName

● Description

- Open된 파일의 Name을 확인 하는 함수

● Syntax

- `const char* GetFileName() const;`

● Parameter

- Nothing

● Return

- 파일의 Name

● Sample

```
TFile Test;
TString FileName;
if(Test.Open( _TEXT("JobHistory.cfg") , TFile::O_READ ) ) {
    FileName.Format(_TEXT("%s"),Test.GetFileName());
}
```

6. IsEOF

- **Description**

- 파일의 끝을 확인 하는 함수

- **Syntax**

- `bool IsEOF() const;`

- **Parameter**

- Nothing

- **Return**

- true : 파일의 끝(end of file)
- false : 파일의 끝이 아님

- **Sample**

7. ReadChar

- **Description**

- Open된 파일의 한 문자를 읽는 함수.

- **Syntax**

- int ReadChar() const;

- **Parameter**

- Nothing

- **Return**

- 문자를 Int 형으로 return

- **Sample**

8. Read

● Description

- Open된 파일에 내용을 Read 하는 함수.

● Syntax

- `bool Read( void* pBuf , int nMaxSize , int *nReadCnt ) const;`

● Parameter

- pBuf : Open 한 파일에서 읽은 포인터
- nMaxSize : Max 로 읽을 문자크기
- nReadCnt : 읽은 문자의 카운트

● Return

- true : 정상적으로 읽은 경우
- false :읽지 못한경우

● Sample

9. ReadLine

● Description

- Open한 파일에서 한 Line을 읽는 함수 .

● Syntax

- bool ReadLine(char* szReturn , int nMaxSize , int* nReadCnt);
- bool ReadLine(TStringA& strLine);

● Parameter

- szReturn: Open 한 파일에서 읽은 문자열
- nMaxSize: Max 로 읽을 문자크기
- nReadCnt: 읽은 문자의 카운트
- strLine : Open 한 파일에서 읽은 문자열

● Return

- true : 정상적으로 읽은 경우
- false :읽지 못한경우

● Sample

10. ReadLineEx

● Description

- Open한 파일에서 Line을 Split으로 구분 하여 읽는 함수.

● Syntax

- bool ReadLineEx(char* szReturn , int nMaxSize , int *nItemCnt , const char* szSplitBy=" \t"/*White Space*/ , char cSep=' ' , const char* szComment="//" , bool fTrim=true);
- bool ReadLineEx(TStringArray& strArray , const char* szSplitBy=" \t"/*White Space*/ , const char* szComment="//" , bool fTrim=true);

● Parameter

- szReturn : Open 한 파일에서 읽은 문자열 포인터
- nMaxSize : Max 로 읽을 문자크기
- nItemCnt : Split 으로 구분되어 읽은 item 갯수
- szSplitBy : Split 할 문자열
- cSep : 구분지을 문자
- szComment : 주석으로 간주할 문자열
- fTrim : ture is left/right trim
- strArray : Split 으로 구분되어 읽혀진 문자열 배열

● Return

- true : 정상적으로 읽은 경우
- false :읽지 못한경우

● Sample

```
TFile file;
if( file.Open( szArgument , TFile::O_READ ) ) {
    TStringArray strTokens;
    while( file.ReadLineEx(strTokens,_TEXT(" \t")) ) {
        if( strTokens.count() == 6 )          printf("true\n");
    }
}
```

11. ReadComplexLine

● Description

- Open한 파일에서 multi Line을 Split으로 구분 하여 읽는 함수.
- 특정 문자열 start/end를 소속 item으로 가지고 있는 multi line read

● Syntax

```
bool ReadComplexLine( char* szReturn , int nMaxSize , int *nItemCnt , const char* szSplitBy=" \t"/*White Space*/ , char cSep=' ' , const char *szAtomS="W" , const char *szAtomE="W" , const char *szLineComment="//" , const char *szCommentBegin="/*" , const char *szCommentEnd="*/" , bool fTrim=true );

bool ReadComplexLine( TStringArray& strArray , const char* szSplitBy=" \t"/*White Space*/ , const char *szAtomS="W" , const char *szAtomE="W" , const char *szLineComment="//" , const char *szCommentBegin="/*" , const char *szCommentEnd="*/" , bool fTrim=true );
```

● Parameter

- szReturn : Open 한 파일에서 읽은 문자열 포인터
- nMaxSize : Max 로 읽을 문자크기
- nItemCnt : Split 으로 구분되어 읽은 item 갯수
- szSplitBy : Split 할 문자열
- szAtomS : 소속 item Start 문자열
- szAtomE : 소속 item End 문자열
- szLineComment : Line comment 문자열
- szCommentBegin : comment begin 문자열
- szCommentEnd : comment end 문자열
- fTrim : true is left/right trim
- strArray : Split 으로 구분되어 읽혀진 문자열 배열

● Return

- true : 정상적으로 읽은 경우
- false : 읽지 못한 경우

● Sample

```
TFile file;
if( file.Open( szArgument , TFile::O_READ ) ) {
    TStringArray strTokens;
    while( file.ReadComplexLine(strTokens,_TEXT(" \t") , _TEXT("\n") , _TEXT("\r") ) ) {
        if( strTokens.count() == 2 ) printf("true\n");
    }
}
```

12. ReadBlock

● Description

- 특정 Block으로 표시 되어 있는 부분을 읽는 함수

● Syntax

```
bool ReadBlock( char* szReturn , int nMaxSize , int *nItemCnt , int *nDepth , const char* szSplitBy=" \t" ,
               char cSep=' ' , char cBlockBegin='{', char cBlockEnd='}' , const char *szAtomS="W" ,
               const char *szAtomE="W" , const char *szLineComment="//" , const char
               *szCommentBegin="/*" , const char *szCommentEnd="*/" , bool fTrim=true );

bool ReadBlock( TStringArray& strArray , int *nDepth , const char* szSplitBy=" \t" , char cBlockBegin='{',
               char cBlockEnd='}' , const char *szAtomS="W" , const char *szAtomE="W" , const
               char *szLineComment="//" , const char *szCommentBegin="/*" , const char
               *szCommentEnd="*/" , bool fTrim=true );
```

● Parameter

- szReturn : Open 한 파일에서 읽은 문자열 포인터
- nMaxSize : Max 로 읽을 문자크기
- nItemCnt : Split 으로 구분되어 읽은 item 갯수
- szSplitBy : Split 할 문자열
- cSep : 구분지을 문자
- cBlockBegin: block Start 문자열
- cBlockEnd : block end 문자열
- szAtomS : 소속 item Start 문자열
- szAtomE: 소속 item End 문자열
- szLineComment : Line comment 문자열
- szCommentBegin : comment begin 문자열
- szCommentEnd : comment end 문자열
- fTrim:ture is left/right trim
- strArray : Split 으로 구분되어 읽혀진 문자열 배열

● Return

- true : 정상적으로 읽은 경우
- false :읽지 못한경우

● Sample

13. WriteChar

- **Description**

- Open한 파일에 Char형으로 한 문자를 write하는 함수

- **Syntax**

- bool WriteChar(char c);

- **Parameter**

- c : 파일에 write 할 문자

- **Return**

- true : 성공 , false : 실패

- **Sample**

14. Write

● Description

- Open한 파일에 문자열을 write 하는 함수

● Syntax

- `bool Write( const char* pBuf , int nMaxSize , int *nWriteCnt );`

● Parameter

- pBuf : 파일에 write 할 문자열
- nMaxSize : 파일에 write 할 문자열 최대 크기
- nWriteCnt : 파일에 write 한 문자열 카운트 반환

● Return

- true : 성공 , false : 실패

● Sample

15. WriteLine

● Description

- Open한 파일에 문자열을 write 하는 함수

● Syntax

- `bool WriteLine( const char* szData );`

● Parameter

- szData : 파일에 write 할 문자열

● Return

-true : 성공 , false : 실패

● Sample

```
TFile file;  
if( file.Open(strLogFileName,TFile::O_APPENDPLUS) ) {  
    file.WriteLine("ABCDEF");  
}
```

16. WriteFormat

● Description

- Open한 파일에 Format 문자열을 write 하는 함수

● Syntax

- `bool WriteFormat( const char* szFormat , ... );`

● Parameter

- szFormat : 파일에 write 할 format 문자열

● Return

-true : 성공 , false : 실패

● Sample

```
TFile file;  
if(file.Open( _TEXT("JobHistory.cfg") , TFile::O_WRITEPLUS ) ) {  
    file.WriteFormat( _TEXT("%d,%d,%d,%d,%d,%d\\n") , 60000 , 60001 , 60002 , 60003 , 60004 , 60000 );  
}
```

17. WriteFormatV

- **Description**

- Open한 파일에 Format 문자열 list을 write 하는 함수

- **Syntax**

- bool WriteFormatV(const char* szFormat , va_list arg);

- **Parameter**

- szFormat : 파일에 write 할 format 문자열
 - arg : list argument

- **Return**

- true : 성공 , false : 실패

- **Sample**

18. Flush

● Description

- Open한 파일의 내용을 전부 없애주는 함수

● Syntax

```
bool Flush();
```

● Parameter

- Nothing

● Return

-true : 성공 , false : 실패

● Sample

```
TFile Test;  
if( Test.Open( _TEXT("JobHistory.cfg") , TFile::O_WRITEPLUS ) ) {  
    Test.Flush();  
}
```

19. Find

● Description

- 파일에서 특정 문자 또는 문자열을 찾는 함수

● Syntax

int Find(char c) const;

int Find(const char* szFind , int size) const;

● Parameter

- c : find 하기 위한 문자
- szFind : find 하기 위한 특정 문자열
- size : find 하기 위한 특정 문자열 size

● Return

- 성공: >0

● Sample

20. CopyFrom

● Description

- 지정해놓은 경로에 있는 파일의 내용을 Open한 파일에 내용을 copy 하는 함수.

● Syntax

- `bool CopyFrom( const char* szSrcFile );`

● Parameter

- `szSrcFile` : 복사하고자 하는 파일 경로

● Return

- `true` : 성공 , `false` : 실패

● Sample

```
TFile Test;  
if( Test.Open( _TEXT("JobHistory.cfg") , TFile::O_WRITEPLUS ) ) {  
    Test.CopyFrom( _TEXT("Test.cfg"));  
}
```

21. CopyTo

● Description

- Open한 파일의 내용을 지정해놓은 경로에 있는 파일에 내용을 복사 하는 함수

● Syntax

- `bool CopyTo( const char* szDestFile );`

● Parameter

- `szDestFile` : 복사하고자 하는 파일 경로

● Return

- `true` : 성공 , `false` : 실패

● Sample

```
TFile Test;  
if( Test.Open( _TEXT("JobHistory.cfg") , TFile::O_WRITEPLUS ) ) {  
    Test.CopyTo( _TEXT("Test.cfg"));  
}
```

URL : <http://www.gesolutions.co.kr/>

(주)게스

우 305-500

대전광역시 유성구 용산동 533-1번지 미건테크노월드Ⅱ C동 440호

전화 : 042-671-3331

G.E.S

#440C-Gate Migun TechnoWorld Ⅱ, 533-1

Yongsan-dong, Yuseong-gu, Daejeon, Korea

Tel : +82-42-671-3331

